Backend PharmaStock — Material de Estudo

Projeto Integrado Multidisciplinar III — ADS UNIP | Preparação para Apresentação

1. O que é o backend e o que ele não é

O backend do PharmaStock é a parte do sistema que **não aparece na tela** — é onde ficam as regras do negócio, as validações e os dados. Ele é composto por código C# organizado em classes.

> **Ponto crítico: não há persistência de dados.** Quando o programa é fechado, tudo que foi cadastrado (produtos, fornecedores, funcionários) é perdido. Os dados vivem apenas na **memória RAM** enquanto o programa está rodando. O PostgreSQL foi modelado e os scripts SQL existem em `backend/database/`, mas a conexão não foi implementada nesta versão — é um protótipo funcional completo.

2. O arquivo .sln (Solution)

O arquivo `estoque_farmacia.sln` é a **solução** — pense nele como uma pasta inteligente que agrupa múltiplos projetos C# relacionados. Ele não contém código. Apenas diz ao Visual Studio quais projetos fazem parte do mesmo sistema.

Projeto	O que é
`estoque_farmacia`	Aplicação console (terminal)
`estoque_farmacia_winforms`	Aplicação com janelas visuais (desktop)
`estoque_farmacia.Tests`	Projeto de testes automatizados (xUnit)

3. O arquivo .csproj (C# Project)

Cada projeto tem seu próprio `.csproj`. Ele é o **arquivo de configuração do projeto** — escrito em XML, diz ao compilador como construir aquele projeto.

Console (.csproj)

```
<OutputType>Exe</OutputType>              <!-- gera um executável de terminal -->
<TargetFramework>net10.0</TargetFramework> <!-- usa .NET versão 10 -->
```

WinForms (.csproj) — detalhe importante

```
<OutputType>WinExe</OutputType>         <!-- executável com janela -->
<UseWindowsForms>true</UseWindowsForms>  <!-- habilita o framework de janelas -->

<!-- Referencia os arquivos do projeto console SEM duplicar -->
<Compile Include="..\estoque_farmacia\Models\Produto.cs" Link="Models\Produto.cs" />
<Compile Include="..\estoque_farmacia\Services\ProdutoService.cs" Link="..." />
```

O WinForms **não copia** os Services e Models — ele referencia os mesmos arquivos do console. Mudar uma validação no `ProdutoService` afeta os dois projetos automaticamente.

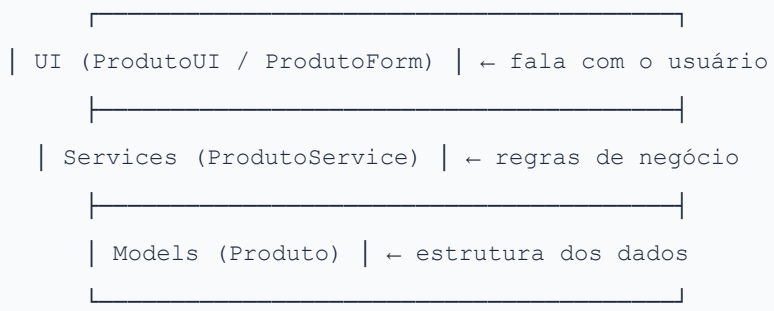
4. O que é Windows Forms

Windows Forms (WinForms) é um **framework da Microsoft para criar aplicações desktop com janelas**. Em vez de digitar texto no terminal, o usuário clica em botões, preenche campos e vê tabelas — como qualquer programa do Windows.

No PharmaStock, cada tela é uma classe chamada `Form`. O WinForms **não tem lógica de negócio própria** — ele chama os mesmos Services do console. A `ProdutoForm.cs` chama `_produtoService.Salvar(produto)` exatamente como a `ProdutoUI.cs` faz no console. A diferença é só a interface.

Arquivo	Tela
`LoginForm.cs`	Janela de login com campos visuais
`MenuForm.cs`	Tela de navegação principal
`ProdutoForm.cs`	Cadastro e listagem de produtos
`VendaForm.cs`	Tela de vendas com carrinho
`VendaHistoricoForm.cs`	Histórico de vendas realizadas
`UIHelper.cs`	Centralizador de cores e estilos visuais

5. As três camadas — o coração da arquitetura



A UI não pode gravar diretamente na lista de produtos — ela passa pelo Service. O Service não sabe se está sendo chamado pelo console ou pelo WinForms. O Model não sabe de nenhum dos dois — ele só guarda os dados.

6. Camada Models

Models são as **classes que representam as entidades do mundo real** no código. Apenas dados — atributos e, no máximo, um método de validação simples.

Classe	Representa
`Produto.cs`	Medicamento com preço, código de barras, fornecedor
`Fornecedor.cs`	Empresa fornecedora com CNPJ e telefone
`Funcionario.cs`	Colaborador com CPF, cargo e senha em hash
`Lote.cs`	Lote de produto com quantidade e validade
`Venda.cs`	Registro de venda com lista de itens (`List<ItemVenda>`)

O que NÃO vai em Models: validações de negócio, consulta a banco, lógica de fluxo. Um Model não sabe se o fornecedor informado existe — isso é responsabilidade do Service.

7. A classe Produto — referência comentada para as demais

`Produto.cs` é a classe mais completa e está **totalmente comentada** para servir de modelo de estudo. Qualquer dúvida sobre como funciona um Model, a resposta está nos comentários dela.

Propriedades (`{ get; set; }`)

```
public int Id { get; set; }           // qualquer um lê e escreve
public string NomeProduto { get; set; }
public decimal PrecoVenda { get; set; }
public bool RequerReceita { get; set; }
```

Uma **propriedade** controla quem lê (`get`) e quem escreve (`set`) um valor. O `{ get; set; }` é a forma curta que libera para todos. Quando você vê `public string UltimoErro { get; private set; }` no Service, significa: qualquer um lê, mas só o próprio Service escreve.

Valor inicial com `string.Empty`

```
public string CodigoBarras { get; set; } = string.Empty;
```

Evita que a propriedade fique `null` quando um objeto é criado sem preencher todos os campos. `string.Empty` é equivalente a `""` .

Método estático

```
public static bool CodigoBarrasValido(string? codigo)
```

`static` significa que esse método pertence à **classe**, não a um objeto. Você chama assim: `Produto.CodigoBarrasValido("789012")` sem precisar criar um `new Produto()` . Faz sentido porque a validação não depende de nenhum produto em particular.

`TryParse` + `out _`

```
return int.TryParse(codigo.Trim(), NumberStyles.Integer, CultureInfo.InvariantCulture, out _);
```

`TryParse` tenta converter uma string para número. Retorna `true` se conseguir, `false` se não. O `out _` descarta o resultado da conversão — aqui só interessa saber se é convertível.

8. Camada Services

Services são as classes com a **lógica de negócio**. Validam, calculam, tomam decisões e guardam os dados em memória.

Service	Responsabilidade
`ProdutoService`	CRUD de produtos, valida fornecedor antes de salvar
`FornecedorService`	CRUD de fornecedores, valida CNPJ único
`FuncionarioService`	CRUD de funcionários, login com SHA-256
`LoteService`	Entrada de lotes, baixa FIFO por validade
`VendaService`	Registro de vendas, baixa de estoque
`Estoque`	Fachada sobre LoteService para consultas de disponibilidade
`SenhaHasher`	Utilitário SHA-256 para hash de senhas

Como os dados ficam em memória

```
private static readonly List<Produto> Itens = new();
private static int _nextId = 1;
```

`List<Produto>` é uma lista dinâmica — cresce conforme você adiciona itens, diferente de um array de tamanho fixo. `static` faz com que seja compartilhada entre todas as instâncias da classe. `readonly` impede que a variável aponte para uma lista diferente após ser inicializada. Quando o programa fecha, a lista some da memória.

Injeção de Dependência no construtor

```
public ProdutoService(FornecedorService fornecedorService)
{
    _fornecedorService = fornecedorService;
}
```

Em vez de criar um `new FornecedorService()` dentro do `ProdutoService`, ele **recebe** o service já criado pelo construtor. Isso é **Injeção de Dependência** — facilita testes e manutenção. Quem monta tudo é o `Program.cs`:

```
var fornecedorService = new FornecedorService();
var produtoService    = new ProdutoService(fornecedorService); // injeta aqui
var loteService       = new LoteService(produtoService);
var estoque           = new Estoque(loteService);
var vendaService      = new VendaService(produtoService, estoque);
```

`try/catch` nos services

```
try { ... }
catch (Exception ex)
{
    UltimoErro = $"ERRO ao salvar produto: {ex.Message}";
    return false;
}
```

Captura qualquer erro inesperado durante a execução. Sem ele, uma exceção não tratada travaria o programa inteiro. Com ele, o método retorna `false` e armazena a mensagem em `UltimoErro`, que a UI pode exibir ao usuário.

Estoque.cs — o padrão Facade

```
public bool TemEstoque(int idProduto, int quantidade)
    => ObterDisponivel(idProduto) >= quantidade;

public bool Baixar(int idProduto, int quantidade)
    => _loteService.BaixarEstoque(idProduto, quantidade);
```

`Estoque` não guarda nenhum dado. Ela só repassa chamadas para o `LoteService` de forma simplificada — isso é o padrão **Facade**. O `VendaService` e a `VendaUI` usam `Estoque` em vez de falar diretamente com o `LoteService`.

FIFO no LoteService

```
var lotes = Itens.Where(l => l.IdProduto == idProduto)
                 .OrderBy(l => l.Validade)  // mais perto do vencimento primeiro
                 .ToList();
```

FIFO (First In, First Out) aplicado por validade: os lotes mais próximos do vencimento são consumidos primeiro. O loop percorre os lotes subtraindo a quantidade pedida, e lotes que chegam a zero são removidos da lista.

SHA-256 no SenhaHasher

```
var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(senha));
// retorna 64 caracteres hexadecimais — irreversível
```

Senhas nunca são gravadas em texto puro. O hash é uma transformação de mão única — não dá para "des-hashear". Se alguém ver a lista de funcionários em memória, vê só o hash, nunca a senha original.

9. Camada UI

Responsável exclusivamente por **interagir com o usuário** — mostrar menus, coletar entradas, exibir resultados. Não tem lógica de negócio própria.

Padrão de loop de menu

```
bool continuar = true;
while (continuar)
{
    MostrarMenuProduto();        // exibe as opções
    string opcao = Console.ReadLine(); // lê a escolha
    switch (opcao)
    {
        case "1": CadastrarProduto(); break;
        case "4": continuar = false; break; // sai do loop
    }
}
```

Validação na entrada com TryParse + while

```
while (!decimal.TryParse(Console.ReadLine(), out precoVenda) || precoVenda < 0)
{
    Console.Write("Valor inválido. Tente novamente: ");
}
```

O loop só avança quando o valor digitado é um número decimal válido e não negativo. `out precoVenda` recebe o resultado da conversão caso seja bem-sucedida.

O objeto é montado na UI, a decisão de salvar é do Service

```
// UI monta o objeto — Id ainda é 0
Produto novo = new Produto
{
    NomeProduto  = nome,
    CodigoBarras = codigoBarras,
    PrecoVenda   = precoVenda,
    // ...
};

// Service decide se é válido e atribui o Id
_produtoService.Salvar(novo);
```

10. Conceitos-chave para a apresentação

Conceito	O que é	Onde no projeto
Propriedade	`{ get; set; }` — controla leitura/escrita	Todos os Models
`private set`	Só a própria classe escreve	`UltimoErro` nos Services
`List<T>`	Lista dinâmica genérica	`List<Produto> Itens` em cada Service
`static`	Pertence à classe, não ao objeto	Listas dos Services, `CodigoBarrasValido`
`readonly`	Não troca após o construtor	Campos de service nas UIs
Injeção de Dependência	Recebe dependência pelo construtor	Todos os Services e UIs
`TryParse`	Converte string para número com segurança	Validações nas UIs
LINQ	`Where`, `FirstOrDefault`, `Any`, `OrderBy`, `ToList`	Services inteiros
`try/catch`	Captura erros sem travar o programa	Métodos dos Services
Facade	Classe que simplifica acesso a outra mais complexa	`Estoque` → `LoteService`
Expression body (`=>`)	Método de uma linha	`ListarTodos()`, `BuscarPorId()`
SHA-256	Hash irreversível de senhas	`SenhaHasher.cs`
FIFO	Primeiro a vencer, primeiro a sair	`LoteService.BaixarEstoque`